

Observability Field Manual

Whole-service observability across Azure, AWS and on-premises: worksheets and queries

Kristopher Turner | CAS26 | October 2026

This is the *do this* document. The session companion explains the ideas; this manual is what you fill in and run. Each section pairs a worksheet with the queries that verify what you just wrote down, so neither is useful without the other and neither makes you flip to a different document to finish the thought. The closing checklist is the one page to keep on the wall.

Every query uses placeholders in capitals, such as `YOUR_RG`; replace them with verified values. No output shown here is a result from your environment. Examples use the fictional IIC Hybrid Orders application (Infinite Improbability Corp is fictional; the Azure, AWS and on-premises services it uses are real) - see the companion for the full description. **Stage-record prerequisite**, used throughout: one record per stage in a custom table with `TimeGenerated`, `CorrelationId`, `Cloud`, `Component`, `Stage`, `Success` (boolean) and `DurationMs` (long). IIC's own values are `Cloud` in `Azure`, `OnPrem`, `AWS`; `Component` in `Coordinator`, `Transport`, `OnPremWorker`, `AwsWorker`; an order is complete only when `Component == 'Coordinator'` writes `Stage == 'Completed'` with `Success == true`. Map your own schema first.

1. Define working

Write this down before you choose agents, tables or dashboards. See companion section 2.

Item	Record
Service and review date	
Customer action	
Expected result	
Time target and objective	
Required dependencies (and where each runs)	
Redundant dependencies	
Evidence that proves success	
Minimum sample before calling it Healthy	
Owner and escalation	

Verify it. Once you have named the customer action above, check whether it already has a result you can query. Order outcomes, counted once per order even though one order touches three clouds:

```
YOUR_STAGE_TABLE_CL
| where TimeGenerated > ago(30m)
| summarize Completed=countif(Component == 'Coordinator'
    and Stage == 'Completed' and Success == true),
    Failed=countif(Success == false),
    FailedIn=make_set_if(Cloud, Success == false),
    LastSeen=max(TimeGenerated) by CorrelationId
| extend Outcome=case(Failed > 0, 'Failed', Completed > 0, 'Completed', 'Unresolved')
| summarize Orders=count(), FailedOrders=countif(Outcome == 'Failed'),
    Unresolved=countif(Outcome == 'Unresolved'), LastSeen=max(LastSeen)
| extend FailurePercent=iff(Orders > 0, 100.0*FailedOrders/Orders, real(null))
```

Zero orders is insufficient evidence, not 0% failure. Remove the last two lines and group by tostring(FailedIn) to see which cloud the failures came from. If nothing here answers "expected result" from the worksheet above, that is the gap to close first, not a signal to configure.

One order's full journey, once you have a correlation ID to chase:

```
YOUR_STAGE_TABLE_CL
| where TimeGenerated > ago(24h)
| where CorrelationId == 'YOUR_CORRELATION_ID'
| project TimeGenerated, Cloud, Component, Stage, Success, DurationMs
| order by TimeGenerated asc
```

The first failed row names the stage and cloud. A missing stage is not a failure yet - check freshness in section 3 before you conclude anything.

2. Evidence routes

Fill one row per route before you trust any dashboard built on it. See companion sections 3 to 5.

Source	Collector	Rule (DCR / DCRA)	Identity	Destination	Expected freshness	Owner

Verify it. Confirm the agent path is bound for one machine, and the direct path has its own endpoint:

```
az monitor data-collection rule association list `
  --resource YOUR_ARC_MACHINE_RESOURCE_ID -o table
```

```
az monitor data-collection rule show -g YOUR_RG -n YOUR_DIRECT_DCR `
  --query "{kind:kind, endpoint:endpoints.logsIngestion, streams:keys(streamDeclarations)}"
```

Expect at least one association naming your DCR, and for the direct rule a `Direct` kind with a logs ingestion endpoint. No association or no matching stream explains a silent route before you suspect the identity.

Then confirm who is actually allowed to send:

```
az role assignment list --scope YOUR_DIRECT_DCR_RESOURCE_ID `
  --include-inherited -o table
```

Expect Monitoring Metrics Publisher for each sender's principal in your table above, and nothing broader than you intend. A sender missing here explains a 403 on ingestion; a broad inherited role explains a sender you did not plan for.

3. Investigation and freshness

Use this when a view looks wrong and you need to know whether the data is late or the service is actually down. See companion sections 6 to 7.

Investigate from customer impact toward narrower evidence:

1. **Confirm impact:** which operation, since when, how many orders.
2. **Scope the order:** one correlation ID and the resources it touched.
3. **Compare stages:** coordinator, on-premises and AWS records for that order.
4. **Inspect dependencies:** the model's state and the freshness of each source.
5. **Decide:** the owner and a bounded action.

Call something a cause only when a test and the evidence support it; stop when the evidence no longer supports the next claim.

Verify it. Freshness per machine:

```
Heartbeat
| where TimeGenerated > ago(30m)
| where _ResourceId =~ 'YOUR_ARC_MACHINE_RESOURCE_ID'
```

```
| summarize LastSeen=max(TimeGenerated), Samples=count() by Computer, Category
| extend EvidenceAge=now()-LastSeen
```

About 30 samples in 30 minutes is normal for AMA. No row means no matching evidence in this workspace and window - check the route in section 2 before you conclude the machine is off.

Freshness per source and stage together, so a late machine and a quiet order path don't look the same:

```
union
  (Heartbeat | where TimeGenerated > ago(1h)
   | summarize LastSeen=max(TimeGenerated) by Source=Computer),
  (YOUR_STAGE_TABLE_CL | where TimeGenerated > ago(1h)
   | summarize LastSeen=max(TimeGenerated) by Source=strcat(Cloud, '/', Component))
| extend EvidenceAge=now()-LastSeen
| order by EvidenceAge desc
```

Compare each age with that source's expected cadence from your section 2 table. Stage records only arrive when orders are placed, so an old stage timestamp can mean no traffic; a synthetic order on a schedule removes that ambiguity.

Ingestion delay, to set alert windows and recovery-wait honestly:

```
YOUR_TABLE
| where TimeGenerated > ago(8h)
| extend E2ELatency=ingestion_time() - TimeGenerated
| summarize percentiles(E2ELatency, 50, 95)
```

Use the 95th percentile as your floor: an alert window or a recovery check set shorter than this will fire on ingestion lag, not on a real problem.

4. Health model design

Complete one row per entity. See companion section 8.

Entity	Parent	Required, redundant or informational	Impact and dependencies setting	Signal (query or metric)	Refresh and time range	Degraded / Unhealthy	When evidence is missing	Owner
--------	--------	--------------------------------------	---------------------------------	--------------------------	------------------------	----------------------	--------------------------	-------

Verify it. Read one entity's current signals and states with the preview health-models CLI extension (az extension add --name health-models):

```
az monitor health-models entity show -g YOUR_RG `
  --health-model-name YOUR_MODEL -n YOUR_ENTITY `
  --query "properties.signalGroups.*.signals[0].{signal:name, state:status.healthState, value:status.value}" `
  -o table
```

Expect every entity that shows a colour above to have at least one named signal or child in your worksheet row. A coloured entity with neither cannot explain itself - fix the worksheet row before you trust the colour.

A worked signal query, for the "Signal" column above - failure percentage over 15 minutes for one entity:

```
YOUR_STAGE_TABLE_CL
| where TimeGenerated > ago(15m)
| where Component == 'AwsWorker' and Stage in ('Completed', 'Failed', 'TimedOut')
| summarize Total=count(), Failed=countif(Success == false)
| project value=iff(Total == 0, real(null), 100.0*Failed/Total)
```

Once the model is live, read its history to see what changed and when - useful evidence when a state looks wrong and you need to know if it is new:

```
az monitor health-models entity get-history -g YOUR_RG `
  --health-model-name YOUR_MODEL --entity-name YOUR_ENTITY `
  --query "history[].[at:occurredAt, from:previousState, to:newState]" -o table
```

5. Cost

Name the question before you cut anything. See companion sections 9 to 10.

Signal	Question it answers	Owner	Grain	Retention	Meter	Accepted blind spot
--------	---------------------	-------	-------	-----------	-------	---------------------

Verify it. Find machines reporting through more than one agent type - the most common source of duplicated cost:

```
Heartbeat
| where TimeGenerated > ago(1h)
| summarize AgentTypes=dcount(Category), Categories=make_set(Category) by Computer
| where AgentTypes > 1
```

Expect no rows. A row showing both Azure Monitor Agent and Direct Agent is a duplicate route to remove from your section 2 table.

Which tables actually drive billable ingestion, before you decide what to cut:

```
Usage
| where TimeGenerated > ago(30d)
| where IsBillable == true
| summarize BillableGB=sum(Quantity) / 1000. by DataType
| order by BillableGB desc
```

Quantity is in MB. This is workspace ingestion, not an invoice; compare equivalent periods before and after a change.

Worker throughput from the metrics store, if you export Prometheus-compatible metrics (Azure Monitor workspace, PromQL - not KQL, and these queries cannot be pasted there):

```
sum by (cloud) (rate(YOUR_ORDERS_COMPLETED_TOTAL[5m]))
```

Metric and label names depend on what your application exports. For scraped targets, `avg_over_time(up[10m]) < 1` lists targets that stopped reporting.

One-page design checklist

1. **Define working.** Customer action, expected result, time target, dependencies and the evidence that proves it (section 1).
2. **Draw the real path.** The application's actual dependencies across every place it runs, drawn separately from Arc management and from telemetry routes. Label what is implemented, designed or conceptual. Do not draw a dependency you have not verified.
3. **Prove each route.** For every signal: source, collector, rule (DCR, and DCRA on agent routes), identity, destination, expected freshness and an owner (section 2).
4. **Correlate on purpose.** One correlation ID per transaction on every stage record. Do not call ID correlation tracing.
5. **Build the view in reading order.** Outcome, health, components, freshness, drill-down, owner. The view visualizes; the model calculates.
6. **Model only what affects the outcome.** Required, redundant and informational dependencies set explicitly; stale or missing evidence shows as Unknown (section 4).
7. **Alert on commitments.** Condition, context, owner and a tested notification.
8. **Rehearse.** A bounded fault on test transactions only, one correlated investigation, an approved action, recovery proven by a fresh transaction after the four clocks catch up.
9. **Pay for questions, not data.** Each high-volume table has a question, owner, plan and retention; duplicate routes removed (section 5).

Review questions. Are all hosting locations included? Can customers complete their work right now, and how fresh is the evidence that says so? Which dependency is affected, and what else depends on it? Who acts, and within what boundary? Did a new transaction prove recovery? Which telemetry could be reduced without losing those answers?

For the explanation behind every item here, see the session companion. Azure Monitor health models are in preview; verify current support, previews, plans and prices before implementation. Reviewed September 26, 2026.

Appendix. What a full observability practice grows into

The session walks one slice of observability on one service. This is the whole map, as commonly listed for public and private cloud observability platforms, with where this session leaves you: **Covered today** (shown on IIC Hybrid Orders), **Next** (the natural next step on the same foundation) or **Later** (part of the practice, not addressed here).

Area	Includes	Status	Where this session leaves you
Infrastructure	Physical servers, virtual machines, hypervisors, containers, operating systems, CPU, memory, disk and network utilisation, storage systems, databases, load balancers, firewalls, routers and switches, DNS and DHCP, VPNs and gateways, backup systems, datacenter facilities, power, cooling and environmental sensors.	Covered today	Heartbeat, performance counters and Syslog from the on-premises and AWS workers through the Azure Monitor Agent; the SCVMM cluster as context. Not covered: network gear, storage, backup, facilities.
Cloud platforms	Azure, AWS, Google Cloud and other providers; subscriptions, accounts and tenants; regions and zones; cloud VMs and managed Kubernetes; serverless; storage; managed databases; queues and messaging; API gateways; load balancers; CDN; virtual networks and peering; cloud security services; costs and usage; quotas and limits; provider service health; configuration drift; tagging and ownership.	Covered today	Azure, AWS and on-premises in one workspace; Azure Arc as the bridge; cost and usage by table. Not covered: quotas, CDN, provider health, other clouds.
Application performance	Availability, response times, throughput, error and request rates, user-experience scores, dependencies, service-to-service calls, background jobs, APIs, web, mobile, serverless and batch workloads, code-level performance, memory leaks, thread and process behaviour, health checks.	Covered today	Availability, response time, error rate and dependencies through the order stage records and Application Insights on the coordinator. Not covered: code-level profiling, mobile.

Area	Includes	Status	Where this session leaves you
Distributed tracing	End-to-end request tracing, spans, cross-service transactions, dependency mapping, database, queue and external API tracing, sampling, correlation with logs and metrics, root-cause analysis across hybrid environments.	Next	Correlation by one ID across the stages, which is not a trace. Next step: OpenTelemetry spans from the coordinator and both workers.
Logs	Application, system, security, audit, network-device, cloud-provider, Kubernetes, database, firewall, proxy, access and error logs; structured logging; collection and forwarding; parsing and enrichment; search; retention and archival; log-based alerting; sensitive-data masking; log cost management.	Covered today	Collection, forwarding, a custom table, log-based signals, retention cost. Not covered: parsing pipelines, masking, security logs.
Metrics and telemetry	Infrastructure, application, business, network, database, Kubernetes, cloud-native, custom, synthetic and user-experience metrics; SLI and SLO measurements; aggregation and correlation; high-cardinality data; telemetry ingestion and normalisation.	Covered today	Infrastructure and application metrics, custom stage records, correlation context, commitments with a time target. Next: formal SLIs, SLOs and error budgets.
Network	Availability, latency, packet loss, jitter, throughput, bandwidth, flow data, east-west and north-south traffic, service mesh, DNS performance, TLS certificates, VPN and private-link health, cloud-to-on-premises connectivity, path analysis, firewall and proxy behaviour.	Later	Only the outbound connectivity of the workers is named. Later: latency, flows, DNS, certificates, private link health between cloud and on-premises.

Area	Includes	Status	Where this session leaves you
Digital experience	Real-user, browser and mobile-app monitoring; synthetic transactions; website availability; page-load performance; user journeys; geographic and device performance; front-end errors; session replay where appropriate; customer-impact analysis.	Covered today	Synthetic transactions (the demo orders are synthetic user journeys) and customer-impact analysis. Not covered: real-user monitoring, browser, session replay.
Databases and data	Availability, query performance, slow queries, locks and deadlocks, connections and pools, replication and failover, storage growth, cache and index performance, backup status, data freshness, ETL and ELT pipelines, warehouse and lake health, data-quality monitoring.	Later	Not part of the example service.
Kubernetes and containers	Cluster and node health, pod status, container restarts, requests and limits, scheduling failures, deployments and rollouts, ingress and service health, persistent volumes, control plane, namespace usage, events, image issues, service-mesh telemetry, autoscaling, multi-cluster visibility.	Next	EKS and Container Apps appear as portal origins with Container insights named. Next: pod, node and rollout health as entities in the model.
Security	Authentication and authorisation events, privileged activity, identity and access monitoring, vulnerability findings, endpoint telemetry, intrusion indicators, firewall events, network anomalies, cloud security posture, configuration changes, policy violations, secret and certificate expiry, compliance evidence, SIEM integration, threat detection and response.	Later	Least-privilege identities for ingestion only. Later: identity and access events, posture, SIEM integration.

Area	Includes	Status	Where this session leaves you
Availability and reliability	Availability and uptime, failover and disaster-recovery monitoring, backup verification, RTO and RPO, capacity thresholds, resilience testing, error budgets, SLIs, SLOs and SLAs, dependency risk analysis.	Covered today	Availability, dependency risk, required versus redundant, recovery proven with fresh evidence. Next: RTO/RPO, DR monitoring, error budgets.
Alerting and events	Threshold, anomaly, forecast, log, synthetic, security and dependency-aware alerts; correlation and deduplication; suppression and maintenance windows; escalation policies; on-call scheduling; acknowledgement and ownership; routing by team, service or severity; noise reduction; alert-quality monitoring.	Covered today	Dependency-aware alerts, one alert per state change, ownership and routing, noise reduction, delivery verification. Next: on-call scheduling and escalation tooling.
Incidents and operations	Incident creation, prioritisation and assignment; on-call and escalation; chat and ticketing integration; change and deployment correlation; incident timelines; runbooks; automated remediation; post-incident reviews; problem management; knowledge base; MTTD, MTTA and MTTR.	Covered today	Investigate, bounded action, recovery proof, runbooks. Next: ticketing integration, incident timelines, MTTD/MTTA/MTTR, post-incident reviews.
Dashboards and reporting	Operational, executive, service and cloud dashboards; infrastructure, dependency and topology maps; geographic views; capacity, availability, SLO and compliance reports; cost and usage dashboards; trend and historical analysis; custom queries.	Covered today	The service view, the health model as a dependency map, cost by table. Next: executive and compliance reporting.

Area	Includes	Status	Where this session leaves you
Automation and remediation	Auto-scaling, service restarts, VM and container remediation, traffic rerouting, failover automation, ticket creation, notification workflows, runbook automation, infrastructure-as-code integration, deployment rollback, automated quarantine and enrichment, event-driven workflows, ChatOps.	Later	Deliberately manual today: bounded action by the owner, verified with fresh orders, before anything is automated.
Configuration and assets	Asset discovery, service and configuration inventory, dependency ownership, CMDB integration, tagging and metadata, environment classification, application-to-infrastructure mapping, drift detection, change tracking, software and hardware inventory, end-of-life tracking.	Next	An inventory of expected sources and dependency ownership. Next: change tracking and drift detection (see the StratoLens call-out for the Azure side).
Integrations	OpenTelemetry, Prometheus, Grafana, OpenMetrics, Fluent Bit or Fluentd, Syslog, SNMP, REST and GraphQL APIs, webhooks, ITSM, SIEM and ticketing platforms, cloud-native monitoring services, CI/CD, infrastructure-as-code tools, collaboration tools, CMDB systems, identity providers.	Later	Azure Monitor native only. Later: OpenTelemetry, Prometheus and Grafana, ITSM and SIEM integration.

Area	Includes	Status	Where this session leaves you
Governance	Role-based access control, multi-tenancy, single sign-on, multi-factor authentication, data-residency controls, encryption in transit and at rest, retention policies, data masking, audit trails, compliance controls, tenant and team separation, delegated administration, usage quotas, data-access policies, licensing and cost controls.	Later	Reader roles on the workspace and data residency named. Later: RBAC design, retention policies, audit trails, tenant separation.
Platform architecture	Telemetry agents, collectors, gateways, ingestion, queues and buffers, time-series, log and trace stores, search and analytics engines, data lakes, long-term archives, high availability, disaster recovery, horizontal scalability, hybrid connectivity, edge and disconnected-environment support.	Covered today	Agents, collectors, the logs ingestion API, one workspace, disconnected sites and Unknown. Not covered: HA and DR of the platform itself.

In one sentence, the whole map is: metrics, logs, traces, events, topology, user experience and security telemetry, connected to alerting, dashboards, incident response, automation, governance and reporting, across on-premises, private cloud, public cloud, edge and hybrid environments. Eleven of the twenty areas are covered today; four are a natural next step on the same foundation; five are later.